Workspace ONE to Intune Application Rebuild & Repackaging

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Workspace ONE to Intune Application Rebuild & Repackaging 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Workspace ONE to Intune Application Rebuild & Repackaging 5

What this solution is . . . 5

Why apps are the hard part of a Workspace ONE migration . . . 5

How Workspace ONE app types map to Intune . . . 5

How the rebuild works . . . 6

What you control per app . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Workspace ONE to Intune Application Rebuild & Repackaging 7

2.1 Intune app-management admin rights . . . 7

2.2 Source app inventory export from Workspace ONE . . . 7

2.3 Windows Win32 Content Prep Tool (for internal Win32 apps) . . . 7

2.4 Original installer media for internal Win32 apps . . . 7

2.5 Per-platform app channels (store/mobile) . . . 7

2.6 Captured detection criteria per Win32 app . . . 8

2.7 Assignment mapping (source groups to Entra groups) . . . 8

2.8 A pilot device set and a pilot Entra group . . . 8

Checklist . . . 8

3. Implementation Guide — Workspace ONE to Intune Application Rebuild & Repackaging 9

Named objects (reuse these names verbatim everywhere) . . . 9

Step 1 — Inventory and map the source app estate (Assess) . . . 9

Step 2 — Build the assignment ring groups (Configure) . . . 10

Step 3 — Rebuild an internal Win32 app — wrap the source . . . 10

Step 4 — (If needed) Rebuild the dependency app first . . . 10

Step 5 — Create the rebuilt main Win32 app in Intune . . . 10

Step 6 — Set install/uninstall commands and behavior . . . 11

Step 7 — Author requirement and detection rules (parity, not copy) . . . 11

Step 8 — Declare dependencies and supersedence . . . 11

Step 9 — Rebuild store and managed mobile apps . . . 12

Step 10 — Recreate app configuration . . . 12

Step 11 — Recreate the assignment workflow (pilot first) . . . 12
Step 12 — Validate each app on a pilot device (the gate) . . . 12
Step 13 — Expand to the broad ring and retire the source assignment . . . 13
5. Testing Guide — Workspace ONE to Intune Application Rebuild & Repackaging 13
Test 1 — App assignment applied . . . 13
Test 2 — App installs on the pilot device . . . 13
Test 3 — Intune reports the app Installed . . . 14
Test 4 — App configuration applied (where applicable) . . . 14
Test 5 — Detection parity (no reinstall loop, no false-installed) . . . 14
Test 6 — Supersedence replaces the prior version . . . 14
Test 7 — Assignment parity across the estate . . . 14
Test 8 — No double-install during coexistence . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Workspace ONE to Intune Application Rebuild & Repackaging 14
A. App not installing on the device . . . 15
B. Detection problems (reinstall loop or false "installed") . . . 15
C. Assignment parity issues . . . 15
D. Store and managed mobile apps . . . 15
E. Double-install or conflict during coexistence . . . 16
F. Win32 repackaging . . . 16
G. Diagnostics quick reference . . . 16
H. When to escalate . . . 16
7. Best Practices — Workspace ONE to Intune Application Rebuild & Repackaging 16
8. Deliverables Summary — Workspace ONE to Intune Application Rebuild & Repackaging 17
Documentation . . . 17
PowerShell scripts . . . 18
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Workspace ONE to Intune Application Rebuild & Repackaging

Apps are where a Workspace ONE to Microsoft Intune migration bogs down. Every Win32, store, and managed mobile app has to be rebuilt in Intune with correct detection, requirements, assignments, and configuration — and validated, so nobody loses the software they work in. Done by hand it is slow, inconsistent, and easy to get wrong.

This package is a repeatable way to rebuild and repackage the entire Workspace ONE application estate into Intune: inventory the source apps, prioritize them, rebuild each with correct logic, recreate the deployment and assignment workflows, handle app configuration, and prove every app still deploys before cutover.

What you get

- A plain-language Solution Overview of the app-migration model — mapping Workspace ONE app types

to Intune (Win32 `.intunewin`, store apps, managed mobile apps), detection, requirement and assignment parity, app configuration, and the per-app validation gate.

- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps in the Intune admin center.
- Five PowerShell scripts for source-vs-target inventory discovery, app install/detection

validation, a rebuild-coverage and deployment-success report, safe remediation of failed installs, and an app-assignment group builder.

- Testing and Troubleshooting guides with explicit pass/fail gates.

What this delivers

A rebuilt application estate in Intune — each app reconstructed with correct detection, requirements, supersedence, app configuration, and assignment workflow parity — piloted and proven to deploy before the fleet cuts over, so no user loses the software they depend on.

The deployment model

Assess to Configure to Pilot to Validate to Expand, with a per-app validation gate that no app passes until it actually installs and is detected on a pilot device.

Outcome

Every Workspace ONE app rebuilt in Intune and proven to deploy — correct detection, requirements, and assignments, validated before cutover — proven on a pilot before you scale.

1. Solution Overview — Workspace ONE to Intune Application Rebuild & Repackaging

What this solution is

This is a repeatable way to rebuild and repackage the application estate currently delivered by VMware Workspace ONE UEM into Microsoft Intune. Applications do not migrate automatically between MDM platforms: an app deployed by Workspace ONE is a package plus a set of delivery rules (detection, requirements, assignment, configuration) that live in the source platform. To move it, you reconstruct the same app and the same rules natively in Intune, then prove it still installs.

Why apps are the hard part of a Workspace ONE migration

Most of a UEM migration is one-to-one (a profile here, an enrollment there). Apps are not. A single line-of-business app can carry a custom installer, a bespoke detection rule, version supersedence, platform-specific requirements, an app-configuration payload, and an assignment graph spanning several groups. Multiply that by hundreds of apps across Windows, iOS/iPadOS, and Android, and the estate becomes the longest, riskiest pole in the migration. The cost of getting it wrong is direct: a user opens their laptop after cutover and the app they bill hours in is gone. This solution exists to make the rebuild systematic and to gate every app behind proof that it deploys.

How Workspace ONE app types map to Intune

Each source app type has a target equivalent in Intune. You decide the mapping per app during the inventory step.

Workspace ONE app type	Intune target	Notes
Internal Win32 app (EXE/MSI)	Windows app (Win32) `.intunewin`	Repackage source media; author detection + requirements
Public Windows app	Microsoft Store app (new)	Reference by Store package; no repackaging
Internal/Public iOS app	iOS store app or line-of-business app (`.ipa`)	Public from the App Store; internal as LOB
Internal/Public Android app	Managed Google Play app	Public or privately-published via managed Google Play
Per-app configuration	App configuration policy (managed apps / managed devices)	Recreate keys/values against the rebuilt app
Assignment / smart group	Microsoft Entra security group assignment	Rebuild required/available/uninstall intent parity

> *Workspace ONE internals (how the source console exports app records, smart groups, and assignment rules) are deferred to VMware documentation. This guide is exact on the Intune side; on the source side it tells you what to obtain, not how the source console produces it.*

How the rebuild works

1. Inventory the source app estate (name, type, version, install/uninstall logic, detection, requirements, assignments, app config) and produce a source-of-truth list.

1. Map each app to its Intune target type and prioritize by criticality and complexity.

2. Rebuild each app in Intune with correct logic: for Win32, repackage to `.intunewin` and author detection, requirements, and supersedence; for store and managed mobile apps, reference the correct store entry; recreate app configuration.

1. Recreate the deployment/assignment workflow — required, available, and uninstall intent — against Entra groups that mirror the source assignment, using consistent ring targeting.

1. Validate each app on a pilot device (it installs and Intune detects it) before expanding.

2. Expand to the broad ring, then decommission the source assignment per the coexistence plan.

What you control per app

• Detection parity — the rule that tells Intune the app is installed (MSI product code, file version, registry value, or store presence) must match reality, not the source platform's logic.

• Requirement parity — architecture, minimum OS, and any custom requirement rule.

• Supersedence — replacing an older rebuilt version cleanly or in place.

• Assignment parity — the same users/devices get the same app, with the same required/available/uninstall intent, via ring-based targeting.

• App configuration parity — managed-app keys and values recreated against the rebuilt app.

Limitations (be honest with buyers)

• Source-side export of app records is a Workspace ONE task and depends on what that platform and its API expose; this solution flags the data you need and defers the how to VMware docs.

• Stored installer media for internal Win32 apps must be available to repackage — if the original source media is lost, it must be re-obtained before rebuild.

• App configuration schemas differ between platforms; values are recreated, not byte-copied.

• During coexistence both platforms may target a device; the validation gate and uninstall-intent planning exist precisely to avoid double-installs and orphaned apps.

The standard deployment model

> *Inventory and map the source apps to Configure (rebuild each in Intune) to Pilot (assign to a pilot ring) to Validate (prove each app installs and is detected) to Expand (broad ring, then retire the source assignment). Model: Assess to Configure to Pilot to Validate to Expand, with a per-app validation gate.*

2. Requirements Guide — Workspace ONE to Intune Application Rebuild & Repackaging

Verify every requirement before rebuilding any app. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites (target tenant, licensing, source access, inventory, enrollment, pilot group, change control) are in `global-requirements-migrations.md` and are assumed — the items below are specific to the application rebuild.

2.1 Intune app-management admin rights

Description: An admin who can create and assign apps in Intune (Intune Administrator, or a custom role with Mobile apps read/create/assign). Why: Every rebuild action — adding apps, authoring detection, assigning groups — requires app write rights in the target. Verify: `https://intune.microsoft.com` to Apps to All apps loads and Add is available. Implement: Assign Intune Administrator or a custom role granting Mobile apps permissions.

2.2 Source app inventory export from Workspace ONE

Description: An export of the in-scope Workspace ONE apps with, per app: name, type, version, install/uninstall logic, detection criteria, requirements, assignment groups and intent, and any app-configuration payload. Why: You cannot rebuild what you have not inventoried; this export is the source of truth the rebuild maps against. Verify: You hold a current source-app export (CSV/JSON) covering every in-scope app. Implement: Produce the export from the Workspace ONE console / API. The exact source-console export procedure is deferred to VMware documentation; run `WS1APP-Discovery.ps1` afterward to compare what already exists in Intune against this list.

2.3 Windows Win32 Content Prep Tool (for internal Win32 apps)

Description: The Microsoft Win32 Content Prep Tool (`IntuneWinAppUtil.exe`) on a packaging workstation, to wrap internal Windows installers into `.intunewin`. Why: Internal EXE/MSI apps must be repackaged to `.intunewin` before Intune can deploy them. Verify: `IntuneWinAppUtil.exe` is present (e.g. `C:\Tools\IntuneWinAppUtil\`). Implement: Download the Win32 Content Prep Tool from Microsoft and place it on the packaging workstation. Not needed for store or managed mobile apps.

2.4 Original installer media for internal Win32 apps

Description: The clean source installer (EXE/MSI) and any silent-install switches for each internal Win32 app being rebuilt. Why: Repackaging to `.intunewin` requires the original media and the silent install/uninstall commands; the source platform stored a copy, not the build recipe. Verify: For each internal app, you have the installer file and its documented silent commands. Implement: Re-obtain the installer and silent switches from the vendor or your software repository.

2.5 Per-platform app channels (store/mobile)

Description: The Intune connections for the platforms in scope — Microsoft Store integration for new Store apps, the Apple VPP/ABM and APNs plumbing for iOS apps, and managed Google Play for Android. Why: Store and managed mobile apps are referenced from these channels, not repackaged; the channel must

be live before those apps can be added. Verify: In Intune, Tenant administration to Connectors and tokens shows the relevant connector/token Active for each platform in scope (also covered by global requirement G5). Implement: Configure the Apple VPP/ABM token, APNs certificate, and managed Google Play connection per the platforms you are rebuilding.

2.6 Captured detection criteria per Win32 app

Description: For each internal Win32 app, the authoritative detection signal — MSI product code, an installed file path and version, or a registry uninstall key plus `DisplayVersion`. Why: Intune detection must reflect what the installer actually leaves on disk, not the source platform's detection record; a wrong rule causes endless reinstall loops or false "installed". Verify: For each app you can state exactly one authoritative detection rule (and its value). Implement: Capture the MSI product code (`Get-Package`), the installed EXE path/version, or the uninstall registry key on a reference machine before building the app.

2.7 Assignment mapping (source groups to Entra groups)

Description: A mapping from each Workspace ONE assignment (smart group) to a Microsoft Entra security group, with the intent (required, available, or uninstall) recorded per app. Why: Assignment parity is what guarantees the same users/devices receive the same app after cutover; without the map you cannot rebuild the deployment workflow. Verify: Each in-scope app has a target Entra group and intent recorded in the inventory. Implement: Create or identify the Entra groups (use `WS1APP-Set-AppAssignmentTargeting.ps1` to build the ring groups) and record the source-to-target group mapping per app.

2.8 A pilot device set and a pilot Entra group

Description: A small set of representative enrolled devices per platform and an Entra pilot group, to validate each rebuilt app before fleet assignment. Why: Every app passes the validation gate on a pilot before it reaches the broad ring. Verify: Entra to Groups shows the pilot group; pilot devices are enrolled in Intune. Implement: Create the pilot group (also global requirement G6) and enroll representative devices.

Checklist

- Intune app-management admin rights
- Source app inventory export from Workspace ONE — `WS1APP-Discovery.ps1`
- Win32 Content Prep Tool on a packaging workstation
- Original installer media + silent commands for internal Win32 apps
- Per-platform app channels (Store / Apple VPP-ABM-APNs / managed Google Play) Active
- Captured detection criteria per Win32 app
- Assignment mapping (source groups to Entra groups) with intent
- Pilot device set and pilot Entra group

3. Implementation Guide — Workspace ONE to Intune Application Rebuild & Repackaging

Follow these steps in order. Inventory and packaging happen on a workstation; everything else is in the Microsoft Intune admin center (`https://intune.microsoft.com`). Source-side export from Workspace ONE is deferred to VMware documentation — this guide tells you what to obtain, then is exact on the Intune side.

> *Model: inventory the source apps, map each to an Intune type, rebuild each with correct detection and requirements, recreate the assignment workflow, then pilot, validate per app, and expand. The per-app validation gate is the rule: no app reaches the broad ring until it installs and is detected on a pilot device.*

Named objects (reuse these names verbatim everywhere)

Every object follows `<Purpose> – <Scope> – <Stage>` so it is self-describing and searchable. The worked example below rebuilds an internal Win32 line-of-business app `Acme Field Suite` (installer `AcmeField.msi`, version 4.2.0), a public Windows store app, and an iOS store app.

Object	Name to create
Pilot ring group (Windows)	`App Rebuild – Pilot Devices – Ring 1`
Broad ring group (Windows)	`App Rebuild – Broad Devices – Ring 2`
Pilot ring group (mobile users)	`App Rebuild – Pilot Users – Ring 1`
Broad ring group (mobile users)	`App Rebuild – Broad Users – Ring 2`
Rebuilt main Win32 app	`Acme Field Suite – Win32 – v4.2.0`
Dependency Win32 app	`VC++ Redistributable x64 – Win32 – Dependency`
Superseded prior rebuilt app	`Acme Field Suite – Win32 – v4.1.0`
Detection rule identifier	`Acme Field Suite – Detection – MSI ProductCode`
Requirement rule	`Acme Field Suite – Requirement – x64 MinOS22H2`
Rebuilt Store app	`Company Reader – Store – Public`
Rebuilt iOS store app	`Company Reader – iOS – Public`
App configuration policy	`Acme Field Suite – AppConfig – Managed`

Step 1 — Inventory and map the source app estate (Assess)

1. Obtain the Workspace ONE source-app export (Requirement 2.2). The source-console export procedure

is deferred to VMware documentation.

1. Run `WS1APP-Discovery.ps1` to enumerate what already exists in Intune (`mobileApps`) and produce

a source-vs-target gap list — apps still to rebuild, by type.

1. For each app, record its Intune target type from the mapping table in the Solution Overview, plus its detection criterion (Requirement 2.6) and assignment mapping (Requirement 2.7).

1. Prioritize: critical line-of-business apps and the most-assigned apps first; one-off or low-criticality apps last.

Step 2 — Build the assignment ring groups (Configure)

1. Entra to Groups to New group to Security to `App Rebuild – Pilot Devices – Ring 1` (a few representative Windows devices). Create. (Or run `WS1APP-Set-AppAssignmentTargeting.ps1`.)

1. Create `App Rebuild – Broad Devices – Ring 2` (the rest of the Windows fleet).

2. For mobile, create `App Rebuild – Pilot Users – Ring 1` and `App Rebuild – Broad Users – Ring 2` (mobile apps are commonly user-targeted). Create.

Step 3 — Rebuild an internal Win32 app — wrap the source

1. On the packaging workstation, confirm the Win32 Content Prep Tool is present (Requirement 2.3) at `C:\Tools\IntuneWinAppUtil\`.

1. Stage the clean source folder `C:\Packaging\AcmeField\` with `AcmeField.msi` inside (Requirement

2.4).

1. Wrap it: `IntuneWinAppUtil.exe -c C:\Packaging\AcmeField -s AcmeField.msi -o C:\Packaging\Output`.

2. Confirm `C:\Packaging\Output\AcmeField.intunewin` is produced — this is the file you upload.

Step 4 — (If needed) Rebuild the dependency app first

If the app needs a runtime, rebuild and create it before the main app so it can be referenced.

1. Wrap the runtime (Step 3) and create a Win32 app named `VC++ Redistributable x64 – Win32 – Dependency` with its own install/uninstall commands and detection rule (its registry key).

1. You do not assign the dependency separately — it auto-installs via the main app's dependency declaration in Step 8.

Step 5 — Create the rebuilt main Win32 app in Intune

1. Intune to Apps to Windows to Add. For App type choose Windows app (Win32). Select.

2. App package file: browse to `C:\Packaging\Output\AcmeField.intunewin`. OK.

3. App information: Name = `Acme Field Suite – Win32 – v4.2.0`; set Publisher and a short Description. Next.

Step 6 — Set install/uninstall commands and behavior

On the Program tab, recreate the source app's silent logic (Requirement 2.4):

1. Install command: `msiexec /i "AcmeField.msi" /qn /norestart`
2. Uninstall command: `msiexec /x {PRODUCT-GUID} /qn /norestart`
3. Install behavior: System (per-device; correct for most line-of-business apps).
4. Device restart behavior: No specific action (the `/norestart` switch handles reboots).
5. Return codes: leave the defaults (`0` and `1707` success, `3010` soft reboot, `1641` hard reboot, `1618` retry) unless the vendor documents others. Next.

Step 7 — Author requirement and detection rules (parity, not copy)

On the Requirements tab:

1. Operating system architecture: 64-bit.
2. Minimum operating system: your floor (e.g. Windows 11 22H2).
3. Add a custom requirement only if needed, named `Acme Field Suite – Requirement – x64 MinOS22H2`.

Next.

On the Detection rules tab, choose Manually configure detection rules (more reliable than auto), then Add the rule you captured in Requirement 2.6, identified as `Acme Field Suite – Detection – MSI ProductCode`:

• Rule type MSI to enter the MSI product code `{PRODUCT-GUID}` (best for MSIs); optionally check the product version.

• Alternative (EXE apps) — Rule type File to path `C:\Program Files\Acme\Field\`, file `AcmeField.exe`, Detection method version, operator greater than or equal to, value `4.2.0`.

• Alternative — Rule type Registry to the app's uninstall key plus a `DisplayVersion` value check.

Use exactly one authoritative detection rule type per app. Next.

Step 8 — Declare dependencies and supersedence

1. Dependencies tab to Add to select `VC++ Redistributable x64 – Win32 – Dependency` to set

Automatically install = Yes so it installs first if missing. Next.

1. Supersedence tab (when replacing an older rebuilt version) to Add to select

`Acme Field Suite – Win32 – v4.1.0` to choose Uninstall previous version = Yes (clean replace) or No (in-place upgrade), per the installer's behavior. Next.

Step 9 — Rebuild store and managed mobile apps

These are referenced, not repackaged.

1. Public Windows app: Apps to Windows to Add to App type Microsoft Store app (new). Search the Store,

select the exact app, name it `Company Reader – Store – Public`. Set install behavior. Next.

1. iOS store app: Apps to iOS/iPadOS to Add to App type iOS store app. Search the App Store, select

the exact app (confirm the exact publisher and bundle ID so you pick the right one, not a similarly-named app), name it `Company Reader – iOS – Public`. Next. (Internal iOS apps use App type Line-of-business app with the `.ipa`.)

1. Android app: Apps to Android to Add to App type Managed Google Play app; approve and select the

app in managed Google Play, then sync.

Step 10 — Recreate app configuration

For managed apps that carried a configuration payload in the source (Requirement 2.2):

1. Apps to App configuration policies to Add to Managed devices (or Managed apps).

2. Name `Acme Field Suite – AppConfig – Managed`; target the platform and the rebuilt app; recreate

the configuration keys and values from the source payload (values are recreated, not byte-copied).

1. Assign to the same groups as the app. Create.

Step 11 — Recreate the assignment workflow (pilot first)

For each rebuilt app, edit Assignments and recreate the source intent — but pilot first:

1. Required: Add group to `App Rebuild – Pilot Devices – Ring 1` (Windows) or

`App Rebuild – Pilot Users – Ring 1` (mobile). Use Available for enrolled devices instead if the source app was optional via a portal.

1. Uninstall: add an Uninstall assignment only where the source workflow removed the app.

2. Do not assign the broad ring yet — the pilot is the gate. Save.

Step 12 — Validate each app on a pilot device (the gate)

1. On a pilot device, sync policy (Settings to Accounts to Access work or school to Info to Sync).

2. Run `WS1APP-Validation.ps1` (Win32: pass the detection type and value) to confirm the app

installed and the detection rule matches.

1. Run `WS1APP-Report.ps1` to confirm Intune reports the app Installed for the pilot devices via the

app `installSummary` and per-device `deviceStatuses`. Complete the Testing Guide.

1. An app passes the gate only when it installs cleanly and Intune detects it on the pilot. Chase any

Failed device with `WS1APP-Remediation.ps1` (default `-WhatIf`).

Step 13 — Expand to the broad ring and retire the source assignment

1. Only after an app passes its validation gate, edit its Assignments to Required to Add group to

`App Rebuild – Broad Devices – Ring 2` (or `App Rebuild – Broad Users – Ring 2`). Save.

1. Re-run `WS1APP-Report.ps1` over the next cycle and chase any Failed devices.

2. Per the coexistence and rollback plan (global requirement G7), retire the Workspace ONE assignment

for that app only after Intune deployment is confirmed healthy — avoiding a double-install during coexistence.

Order of operations

```
Inventory + map source apps -> Ring groups -> Wrap .intunewin (Win32) ->
Dependency app -> Main Win32 app -> Commands -> Requirements + detection ->
Dependencies + supersedence -> Store/mobile apps -> App configuration ->
Assign pilot -> Validate per app (gate) -> Expand broad -> Retire source
assignment
```

5. Testing Guide — Workspace ONE to Intune Application Rebuild & Repackaging

Run these per app after pilot assignment (Implementation Step 12) and again after broad expansion. The per-app validation gate is the rule: an app does not move to the broad ring until it passes Tests 1 to 4 on a pilot device.

Test 1 — App assignment applied

Do: Intune to Apps to All apps to open the rebuilt app (e.g. `Acme Field Suite – Win32 – v4.2.0`) to Properties to Assignments. Confirm the pilot group is assigned with the correct intent. Expected: The pilot group (`App Rebuild – Pilot Devices – Ring 1` or `App Rebuild – Pilot Users – Ring 1`) is present with Required/Available matching the source workflow.

Test 2 — App installs on the pilot device

Do: On a pilot device, sync policy and let the app install; or run `WS1APP-Validation.ps1` with the app's detection type and value. Expected: For Win32, the app is present and the detection rule matches (MSI product code, file version, or registry value). For store/mobile apps, the app appears installed on the device.

Test 3 — Intune reports the app Installed

Do: Open the app to Device install status (or Overview); or run `WS1APP-Report.ps1`. Expected: The pilot devices show Installed in the app `installSummary` / per-device `deviceStatuses`; no Failed devices in the pilot.

Test 4 — App configuration applied (where applicable)

Do: For apps with an app configuration policy (`Acme Field Suite – AppConfig – Managed`), confirm the configured behavior on the pilot device (the app reflects the configured keys/values). Expected: The app behaves per the recreated configuration — parity with the source payload.

Test 5 — Detection parity (no reinstall loop, no false-installed)

Do: After install, re-sync and observe the app status across two cycles. Expected: The app stays Installed without reinstalling each cycle (detection rule is correct) and is not falsely reported Installed when absent.

Test 6 — Supersedence replaces the prior version

Do: For a superseding app, confirm a device on the prior version moves to the new version per the chosen Uninstall-previous setting. Expected: The device ends on the new rebuilt version; the prior version is removed or upgraded as configured.

Test 7 — Assignment parity across the estate

Do: Run `WS1APP-Report.ps1` to list rebuilt apps and their assignments; compare against the source assignment mapping (Requirement 2.7). Expected: Each in-scope app is assigned to the intended group(s) with the intended intent — the same users/devices receive the same apps as in Workspace ONE.

Test 8 — No double-install during coexistence

Do: On a pilot device still managed by both platforms, confirm the app is delivered once and not fighting between platforms. Expected: A single healthy install; the source assignment is retired only after the Intune install is confirmed (Implementation Step 13).

Pass/fail gate

Gate	Requirement
Per-app pilot pass	Tests 1 to 4 pass for that app on a pilot device; no Failed pilot device
Estate broad pass	Tests 3, 5, 7 pass for the broad ring; assignment parity confirmed across in-scope apps before retiring source assignments

6. Troubleshooting Guide — Workspace ONE to Intune Application Rebuild & Repackaging

Use `WS1APP-Validation.ps1` for local install/detection state, `WS1APP-Discovery.ps1` for the source-vs-target gap, `WS1APP-Report.ps1` for fleet deployment status (Microsoft Graph), and `WS1APP-Remediation.ps1` (default `-WhatIf`) for safe fixes.

A. App not installing on the device

Symptom	Cause	Remediation
App never installs	Device not synced, or not in the assigned group	Run `WS1APP-Remediation.ps1`; confirm group membership and the assignment intent
Win32 install fails	Wrong silent install command or return-code handling	Recreate the source silent command; verify return codes on the Program tab
Dependency missing	Runtime not rebuilt or not auto-installed	Rebuild the dependency app first; set Automatically install = Yes in the main app's Dependencies tab

B. Detection problems (reinstall loop or false "installed")

Symptom	Cause	Remediation
App reinstalls every cycle	Detection rule does not match what the installer leaves	Re-capture the MSI product code / file version / registry value and fix the detection rule (one authoritative rule)
Reported Installed but app absent	Detection rule too loose (matches an unrelated path/key)	Tighten the rule to a unique, authoritative signal
Reported Failed but app present	Detection signal differs from the build (e.g. wrong product code)	Align the detection value to the actual installed app

C. Assignment parity issues

Symptom	Cause	Remediation
Wrong users/devices get the app	Source-to-Entra group mapping wrong	Correct the Entra group mapping (Requirement 2.7); re-run `WS1APP-Report.ps1` to confirm parity
App not removed when expected	Missing Uninstall assignment	Add an Uninstall assignment where the source workflow removed the app

D. Store and managed mobile apps

Symptom	Cause	Remediation
Wrong store app selected	Similar-named apps in the store	Reselect by exact publisher and bundle/package ID
iOS/Android app will not deploy	Platform channel not Active	Confirm Apple VPP/ABM + APNs or managed Google Play is Active (Requirement 2.5 / global G5); re-sync

E. Double-install or conflict during coexistence

- A device managed by both platforms is served the app twice. Retire the Workspace ONE assignment for

that app only after the Intune install is confirmed healthy (Implementation Step 13); validate one authority delivers the app.

F. Win32 repackaging

- `.intunewin` build fails or uploads but never installs. Confirm the source folder is clean and the

setup file named in the wrap matches the install command; re-wrap with the Win32 Content Prep Tool (Requirement 2.3) from the original media (Requirement 2.4).

G. Diagnostics quick reference

```
Get-Package | Where-Object { $_.Name -like '*Acme*' }    # confirm an MSI/EXE app is present + its version
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*' | Select-Object DisplayName
Start-Process -FilePath "$env:WINDIR\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM"   # MD
```

Intune-side: open the app to Device install status to read per-device result, or run `WS1APP-Report.ps1`.

H. When to escalate

If the assignment is correct, the device is enrolled and synced, the detection rule matches a manual install, and the app still will not deploy, collect the Intune Management Extension logs (`C:\ProgramData\Microsoft\IntuneManagementExtension\Logs`) for Win32 apps and open a Microsoft support case. Source-side app-export questions are deferred to VMware documentation.

7. Best Practices — Workspace ONE to Intune Application Rebuild & Repackaging

- Gate every app on a pilot. No app reaches the broad ring until it installs and Intune detects it on

a pilot device — the single most important practice in an app rebuild.

- Rebuild detection from reality, not the source record. Author each Intune detection rule from what

the installer actually leaves on disk (MSI product code, file version, or registry value), not from the Workspace ONE detection entry. A wrong rule causes reinstall loops or false "installed".

• Use exactly one authoritative detection rule per app. Multiple rules invite ambiguity; pick the most reliable signal and use it alone.

• Prioritize by criticality and reach. Rebuild critical line-of-business apps and the most-assigned apps first; one-off apps last. This front-loads the risk reduction.

• Name everything with `<Purpose> – <Scope> – <Stage>`. Consistent, self-describing names for apps, detection rules, requirement rules, and ring groups keep a large estate searchable and auditable.

• Recreate assignment intent exactly. Required, available, and uninstall intent must match the source workflow so the same users/devices get the same apps — assignment parity is what users feel.

• Rebuild dependencies before the apps that need them. Create the runtime app first and auto-install it via the main app's dependency declaration, rather than assigning runtimes separately.

• Plan supersedence deliberately. Choose uninstall-previous (clean replace) or in-place upgrade per the installer's real behavior; do not guess.

• Avoid double-installs during coexistence. Retire the Workspace ONE assignment for an app only after the Intune deployment is confirmed healthy, so a device managed by both platforms is not served the app twice.

• Keep the source installer media and silent switches. Repackaging needs the original media; store it in a software repository so a future rebuild does not stall hunting for the installer.

• Report and chase laggards every cycle. Track install status with `WS1APP-Report.ps1` and remediate Failed devices with `WS1APP-Remediation.ps1` (default `-WhatIf`) before expanding.

• Document the mapping. Record the source-to-target app type and group mapping, detection rule, and assignment intent per app for audits and repeatability.

8. Deliverables Summary — Workspace ONE to Intune Application Rebuild & Repackaging

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The app-migration model: source-to-Intune type mapping, detection/requirement/assignment parity, app configuration, and the validation gate
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Migrations)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered steps: inventory, rebuild Win32/store/mobile, app config, assignment workflow, pilot, validate, expand
5. Testing Guide	Per-app validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`WS1APP-Discovery.ps1`	Read-only. Enumerates Intune apps (`mobileApps`) by type and produces a source-vs-target gap list of apps still to rebuild
`WS1APP-Validation.ps1`	Read-only. Confirms a rebuilt app installed and its detection rule matches on the local device
`WS1APP-Report.ps1`	Read-only. Rebuild-coverage and deployment-success report from Intune `installSummary` / `deviceStatuses` (Microsoft Graph)
`WS1APP-Remediation.ps1`	Safe remediation of failed app installs (default `-WhatIf`); forces an MDM sync and re-evaluation
`WS1APP-Set-AppAssignmentTargeting.ps1`	Builds the pilot/broad app-assignment Entra groups for ring targeting (Microsoft Graph)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand, with a per-app validation gate.

Outcome

Every Workspace ONE app rebuilt in Intune and proven to deploy — correct detection, requirements, and assignments, validated before cutover — so no user loses the software they work in.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WS1APP-Discovery.ps1

Purpose: Read-only. Lists the apps already present in Microsoft Intune (mobileApps) grouped by type, and — when given the Workspace ONE source-app export — reports which source apps are NOT yet rebuilt in Intune (the gap to close). Source-side export from Workspace ONE is deferred to VMware documentation; supply that export as a CSV with a 'Name' column via -SourceCsv. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1APP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\WS1APP-Discovery.ps1 -SourceCsv C:\Temp\ws1-apps.csv -CsvPath C:\Temp\gap.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A count of Intune apps by type and, if -SourceCsv is supplied, a list of source apps still to rebuild (optionally exported to -CsvPath). .NOTES Workplace Marketplace — Migrations. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$SourceCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1APP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\WS1APP-Discovery.ps1 -SourceCsv C:\Temp\ws1-apps.csv -CsvPath C:\Temp\gap.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A count of Intune apps by type and, if -SourceCsv is supplied, a list of source apps still to rebuild (optionally exported to -CsvPath). .NOTES Workplace Marketplace — Migrations. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$SourceCsv, [string]$CsvPath)

WS1APP-Remediation.ps1

Purpose: For a pilot/broad device where a rebuilt app shows Failed or Pending, this forces an MDM sync so Intune re-evaluates app assignments and re-attempts the install, then restarts the Intune Management Extension service that drives Win32 app delivery. SAFE BY DEFAULT: runs in -WhatIf mode and only reports the actions; pass -WhatIf:$false to actually run them. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\WS1APP-Remediation.ps1 Execute : powershell -ExecutionPolicy Bypass -File .\WS1APP-Remediation.ps1 -WhatIf:$false Run elevated to restart the Intune Management Extension service. EXPECTED RESULTS In preview, the actions it WOULD take. When executed, an MDM sync is requested and the Intune Management Extension re-attempts delivery; re-run WS1APP-Validation.ps1 after a cycle. .NOTES Workplace Marketplace — Migrations. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param()

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\WS1APP-Remediation.ps1 Execute : powershell -ExecutionPolicy Bypass -File .\WS1APP-Remediation.ps1 -WhatIf:$false Run elevated to restart the Intune Management Extension service. EXPECTED RESULTS In preview, the actions it WOULD take. When executed, an MDM sync is requested and the Intune Management Extension re-attempts delivery; re-run WS1APP-Validation.ps1 after a cycle. .NOTES Workplace Marketplace — Migrations. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param()

WS1APP-Report.ps1

Purpose: Read-only. For each rebuilt app in Intune, reports the deployment outcome from the app installSummary (installed / failed / pending / not-installed counts) so you can see which apps have passed

and which devices still need attention. Optionally drills into per-device results (deviceStatuses) for a single app via -AppId. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WS1APP-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WS1APP-Report.ps1 -AppId -CsvPath C:\Temp\app-devices.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app install summary table; with -AppId, a per-device install-status list for that app. .NOTES Workplace Marketplace — Migrations. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$AppId, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WS1APP-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WS1APP-Report.ps1 -AppId <appId> -CsvPath C:\Temp\app-devices.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app install summary table; with -AppId, a per-device install-status list for that app. .NOTES Workplace Marketplace — Migrations. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$AppId, [string]$CsvPath)

WS1APP-Set-AppAssignmentTargeting.ps1

Purpose: Builds the ring-targeting Microsoft Entra security groups used to assign rebuilt apps, named with the - - convention so they match the Implementation Guide: App Rebuild - Pilot Devices - Ring 1 App Rebuild - Broad Devices - Ring 2 App Rebuild - Pilot Users - Ring 1 App Rebuild - Broad Users - Ring 2 It only CREATES the groups (idempotent: skips any that already exist). Adding members and attaching app assignments are done deliberately in the Intune console per the guide. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\WS1APP-Set-AppAssignmentTargeting.ps1 Execute : powershell -ExecutionPolicy Bypass -File .\WS1APP-Set-AppAssignmentTargeting.ps1 -WhatIf:$false Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The four ring groups exist in Entra (created if missing, reported if already present). .NOTES Workplace Marketplace — Migrations. Creates groups only; assignment is done in the console. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param()

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\WS1APP-Set-AppAssignmentTargeting.ps1 Execute : powershell -ExecutionPolicy Bypass -File .\WS1APP-Set-AppAssignmentTargeting.ps1 -WhatIf:$false Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The four ring groups exist in Entra (created if missing, reported if already present). .NOTES Workplace Marketplace — Migrations. Creates groups only; assignment is done in the console. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param()

WS1APP-Validation.ps1

Purpose: Read-only. Checks that a rebuilt Win32 app is present on THIS device using the same authoritative detection signal you authored in Intune (MSI product code, installed file version, or registry uninstall key + DisplayVersion). Use it on a pilot device as the per-app validation gate before expanding to the broad ring. HOW TO RUN MSI : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType MSI -ProductCode "{PRODUCT-GUID}" File : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType File -FilePath "C:\Program Files\Acme\Field\AcmeField.exe" -MinVersion 4.2.0 Registry : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType Registry -DisplayName "Acme Field Suite" -MinVersion 4.2.0 EXPECTED RESULTS "DETECTED" (exit 0) when the app matches the detection rule; "NOT DETECTED" (exit 1) otherwise. .NOTES Workplace Marketplace — Migrations. Read-only. #> [CmdletBinding()] param(

[Parameter(Mandatory)][ValidateSet('MSI', 'File', 'Registry')][string]$DetectionType, [string]$ProductCode, [string]$FilePath, [string]$DisplayName, [string]$MinVersion)

How to run: MSI : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType MSI -ProductCode "{PRODUCT-GUID}" File : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType File -FilePath "C:\Program Files\Acme\Field\AcmeField.exe" -MinVersion 4.2.0 Registry : powershell -ExecutionPolicy Bypass -File .\WS1APP-Validation.ps1 -DetectionType Registry -DisplayName "Acme Field Suite" -MinVersion 4.2.0 EXPECTED RESULTS "DETECTED" (exit 0) when the app matches the detection rule; "NOT DETECTED" (exit 1) otherwise. .NOTES Workplace Marketplace — Migrations. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][ValidateSet('MSI', 'File', 'Registry')][string]$DetectionType, [string]$ProductCode, [string]$FilePath, [string]$DisplayName, [string]$MinVersion)